

05-ansible 的变量

1 定义变量的注意事项

1. 变量名组成是由 数字、下划线、字母组成
2. 大小写严格区分
3. 变量名必须是字母开头
4. 不能以 ansible 开头作为变量（因为 ansible 具备内置变量，内置变量以 ansible 开头的）

2 定义变量和引用变量的方式

2.1 测试和引用变量

学到一个新的模块，debug 模块

具备 2 个参数：

- var：后面直接写变量名。 变量名
- msg：后面写什么，屏幕上就会输出什么。"{{ 变量名 }}" 来输出变量的值

2 个参数不能同时使用

引用变量：**"{{ 变量名 }}"**

2.2 在主机清单中定义变量

- 定义主机变量
- 定义主机组变量

一般在主机清单中定义的变量分为俩类：

1. 用户自定义变量

定义主机变量，只有对应的主机才能够使用这个变量

Bash 复制

```
node1 pkg=httpd service=nginx
node2 pkg=httpd service=nginx
```

定义主机组变量，只有对应主机组的主机成员才能够使用这个变量

Ini 复制

```
[servers]
node1
node2

[servers:vars]
memeda=huawei
```

如果主机变量和主机组变量冲突，则主机变量优先级更高

2. anisble 内置变量

一般都是 ansible_开头的变量，这些变量是 ansible 自带的，他们的作用和 ansible.cfg 配置文件中的配置参数作用一样

#关于连接的内置变量

YAML 复制

```
ansible_host      #用于指定被管理的主机的真实IP
ansible_port      #用于指定连接到被管理主机的ssh端口号，默认是22
ansible_user      #ssh连接时默认使用的用户名
```

Markdown 复制

```
# 特权升级
ansible_become      #相当于ansible_sudo或者ansible_su，允许强制特权升级
ansible_become_user  #通过特权升级到的用户，相当于ansible_sudo_user或者ansible_su_user
ansible_become_pass  # 提升特权时，如果需要密码的话，可以通过该变量指定，相当于ansible_sudo_pass或者ansible_su_pass
ansible_sudo_exec     #如果sudo命令不在默认路径，需要指定sudo命令路径

# 特定ssh连接
ansible_connection   #SSH连接的类型：local, ssh, paramiko，默认是ssh
ansible_ssh_pass      #ssh连接时的密码
ansible_ssh_private_key_file  #密钥文件路径，如果不想使用ssh-agent管理密钥文件时可以使用此选项
ansible_ssh_executable      #如果ssh指令不在默认路径当中，可以使用该变量来定义其路径
```

ansible 的内置变量是无法通过 ansible 工具查询的，只能够在 ansible 的官网去查询

2.3 在 playbook 中定义变量

2.3.1 通过 vars 关键字定义变量

YAML 复制

```
- name: var
  hosts: all
  vars:
    os1: rhel1
    os2: rhel2
    os3: rhel3
  tasks:
- name: use var
  debug:
    msg: "{{ os1 }} {{ os2 }} {{ os3 }}"
```

上面定义变量还是比较基础的，下面这种方式定义的是变量的嵌套

YAML 复制

```
- name: var
  hosts: all
  vars:
    os1: rhel1
    os2: rhel2
    os3: rhel3
    users:
      username: memeda001
      uid: 3000
      home: /home/memeda001
      shell: /bin/bash
  tasks:
- name: create user
  user:
    name: "{{ users.username }}"
    uid: "{{ users.uid }}"
    home: "{{ users.home }}"
    shell: "{{ users.shell }}"
    state: present
```

这个时候如果直接引用 users 变量的话，则它的值就是 username、uid、home，如果想要引用指定的值，那么使用 变量名.变量名来引用

看下面的案例

YAML 复制

```
- name: var
  hosts: all
  vars:
    os1: rhel1
    os2: rhel2
    os3: rhel3
```

```
tasks:
- name: create user
  debug:
    msg: "{{ os1 }}"

- name: var2
  hosts: all
  tasks:
- name: use var
  debug:
    msg: "{{ os1 }}"
```

然后执行 playbook，会发现 var2 这个 play 执行报错，报错信息如下

YAML 复制

```
fatal: [node1]: FAILED! => {"msg": "The task includes an option with an undefined variable. The error was: 'os1' is undefined\n\nThe error appears to be in '/etc/ansible/var1.yaml': line 16, column 5, but may\nbe elsewhere in the file depending on the exact syntax problem.\n\nThe offending line appears to be:\n\n  tasks:\n    - name: use var\n      ^ here\n"}
```

说明找不到 os1 变量，说明 play 之间定义的变量是不能够通用的

2.3.2 通过 vars_files 关键字引用外部变量文件

如果所有的变量都是通过 vars 在 playbook 中定义的话，那么变量如果数量很多的话，就会导致 playbook 看起来非常的臃肿

如果引用的变量是比较私密的，比如说通过变量的方式来给用户设置密码，如果你写入到了 playbook 中，那么任何人都可以看到这个变量的值就知道了用户的密码，容易泄露

如果是采用引用外部变量文件的方式来加载变量，有一个好处在于我们是可以通过 ansible-vault 命令来对于变量文件加密的，当执行 playbook 的时候 如果你调用了这个外部变量文件的变量，是需要解密才能够执行的

YAML 复制

```
- name: var
  hosts: all
  vars_files:
  - vars.yaml
  - vars1.yaml
  tasks:
- name: create user
  debug:
    msg: "{{ os1 }}"

- name: var2
  hosts: all
  tasks:
- name: use var
  debug:
    msg: "{{ os1 }}"
```

实际上，不需要-，但是不加-的话，只能去引用一个外部变量文件，但是一个 playbook 的项目，有多个外部变量文件都得引用，那么最好在每个变量文件的前面加上一个-

2.4 通过命令行来定义变量

无论是 ansible 命令，还是 ansible-playbook 命令都可以通过一个-e 选项来定义变量

通过选项定义的变量是所有变量中优先级最高的！

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m debug -a 'msg="{{ os1 }}"' -e os1=nihao
[root@adsl-172-10-0-153 ansible]# ansible-playbook var1.yaml -e os1=abcd
```

红帽的很多产品部署都是基于 ansible 实现的，这个产品有不同的版本，红帽通过一个 playbook 就实现的多个版本的安装

通过变量实现的，比如那个 Ceph 分布式存储。部署的时候什么都不加，默认部署的就是最新版本

如果想要部署指定版本的话，可以通过执行剧本的时候使用-e 选项

2.5 通过 `hosts_vars` 和 `groups_vars` 目录定义变量

`hosts_vars` 目录

`groups_vars` 目录

这俩个目录下你创建的文件就是主机或者主机组的名字，这些文件中定义的变量就是给对应主机或者主机组使用的

比如 `host_vars` 目录下存在一个叫做 `node1` 文件，你在 `node1` 文件中定义的变量最终是给到 `node1` 主机使用

比如 `groups_vars` 目录下存在一个叫做 `servers` 文件，你在 `servers` 文件定义的变量最终是给到 `servers` 主机组使用

目录要存放到和 `ansible.cfg` 配置文件同一个地方

比如`/etc/ansible/ansible.cfg`，那么目录就要存储到`/etc/ansible`

Markdown复制

```
/etc/ansible/host_vars:
total 8
-rw-r--r--. 1 root root  9 Apr 19 11:47 node1
-rw-r--r--. 1 root root 10 Apr 19 11:47 node2
```

YAML复制

```
[root@adsl-172-10-0-153 host_vars]# cat /etc/ansible/host_vars/node1
var1: os
[root@adsl-172-10-0-153 host_vars]# cat /etc/ansible/host_vars/node2
var1: aaa
```

2.6 通过 `register` 关键字注册变量

这个通常是在 `playbook` 中使用的，和模块的层级关系是一样的

`register` 关键字可以将一个 `task` 任务的执行结果保存为一个变量—>它都是配合后面的条件判断语句来执行 `task` 任务的

比如 `register` 关键字将前面的一个 `task` 任务执行结果保存为变量，然后后面的 `task` 任务如果要执行的话则判断注册变量的 `rc` 返回值是不是 0

YAML复制

```
[root@adsl-172-10-0-153 ansible]# cat var1.yaml
- name: var
  hosts: all
  vars_files:
  - vars.yaml
  tasks:
  - name: install nginx pkg
    shell: cat /etc/passwd
    register: user_status

- name: shuchu
  debug:
    msg: "{{ user_status }}"
```

需要注意一下，`register` 注册的变量输出结果并不是完全一直的，这里我采用命令执行模块来输出

JSON复制

```
ok: [node1] => {
  "msg": {
    "changed": true,
    "cmd": "cat /opt/passwd",
    "delta": "0:00:00.002148",
    "end": "2026-04-19 11:56:04.439161",
    "failed": false,
    "rc": 0,      # 任务的返回值，如果是0的话说明任务执行成功，非0表示失败
    "start": "2026-04-19 11:56:04.437013",
    "stderr": "",
    "stderr_lines": [],
    "stdout": "root:x:0:0:root:/root:/bin/bash\nbin:x:1:1:bin:/bin:/sbin/nologin\ndaemon:x:2:2:daemon:/sbin:/sbin/nologin\nadm:x:3:4:adm:/var/adm:/sbin/nologin\nlp:x:4:7:lp:/var/spool/lpd:/sbin/nologin",
    "stdout_lines": [
```

```
    "root:x:0:0:root:/root:/bin/bash",
    "bin:x:1:1:bin:/bin:/sbin/nologin",
    "daemon:x:2:2:daemon:/sbin:/sbin/nologin",
    "adm:x:3:4:adm:/var/adm:/sbin/nologin",
    "lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin"
  ]
}
```

通过 register 配合判断语句实现一个条件判断：

当/opt/passwd 文件存在的时候， 才会输出 ok

YAML 复制

```
[root@adsl-172-10-0-153 ansible]# cat var1.yaml
- name: var
  hosts: all
  tasks:
    - name: use shell module
      shell: ls /opt/passwd
      register: status
      ignore_errors: yes

- name: shuchu
  debug:
    msg: ok
  when: status.rc == 0
```

3 fact 事实变量

所谓的 fact 实时变量，其实就是收集了被控节点的主机信息，将其保存到一个叫做 ansible_fats 变量中，这个变量我们称之为 fact 事实变量

默认情况下，只要执行了 playbook，那么 play 中会存在一个隐藏的任务 [gathering facts]，专门去获取节点的主机信息，play 在哪些主机上运行，则获取哪些主机的信息

YAML 复制

```
# plays will gather facts by default, which contain information about
# the remote system.
#
# smart - gather by default, but don't regather if already gathered
# implicit - gather by default, turn off with gather_facts: False
# explicit - do not gather by default, must say gather_facts: True
gathering = implicit

# This only affects the gathering done by a play's gather_facts directive,
# by default gathering retrieves all facts subsets
# all - gather all subsets
# network - gather min and network facts
# hardware - gather hardware facts (longest facts to retrieve)
# virtual - gather min and virtual facts
# facter - import facts from facter
# ohai - import facts from ohai
# You can combine them using comma (ex: network,virtual)
# You can negate them using ! (ex: !hardware,!facter,!ohai)
# A minimal set of facts is always gathered.
gather_subset = all
```

这一段定义了 fact 事实变量的相关信息，之所以默认就收集是这里来的

- facts数据中经常获取的信息如下：

关键字	说明
ansible_python_version	python版本
ansible_distribution	显示是什么系统
ansible_distribution_major_version	显示是系统主版本号
ansible_devices	显示磁盘设备信息
ansible_lvm	显示lvm相关信息
ansible_memtotal_mb	显示系统总内存
ansible_kernel	显示内核版本
ansible_ens160	显示网卡版本
ansible_fqdn	显示主机fqdn
ansible_hostname	显示主机名

fact 事实变量通常都是配合条件判断使用的

模块：setup

如果想要手动收集主机信息，可以利用 setup 模块来实现，setup 模块就是收集信息保存到 ansible_facts 变量中

Bash 复制

```
ansible 主机名 -m setup > 文件
```

将主机的所有信息保存到文件中，你直接去文件中过滤你想要的关键字就可以了

- 关闭和开启 fact 变量收集

YAML 复制

```
- name: var
  hosts: all
  gather_facts: False
  tasks:
    - name: debug
      debug:
        msg: "{{ ansible_default_ipv4.interface }}" "{{ ansible_default_ipv4.address }}"
```

关闭 fact 收集的话，你的 task 任务中就不要引用 fact 变量，因为没有收集，所以变量是不存在的

YAML 复制

```
[root@adsl-172-10-0-153 ansible]# cat var1.yaml
- name: var
  hosts: all
  gather_facts: False
  tasks:
    - name: setup
      setup:

- name: debug
  debug:
    msg: "{{ ansible_default_ipv4.interface }}" "{{ ansible_default_ipv4.address }}"
```

这个案例中，在 target 部分已经关闭了 fact 收集，但是因为 tasks 任务列表中存在一个调用 setup 模块的 task 任务，所以依旧是会收集 fact 变量的

3.1 set_fact 模块整合新的变量

YAML 复制

```
- name: var
  hosts: all
```

```
tasks:
- name: setfact
  set_fact:
    os_version: "{{ ansible_distribution }} {{ ansible_distribution_major_version }}"
- name: use debug
  debug:
    msg: "{{ os_version }}"
```

3.2 lookup 插件，从外部数据源读取内容作为变量的值

有的时候，我们定义的变量，它的值是来自于控制节点的文件的内容的，可能也是来自于控制节点的环境变量的，甚至可能是来自于控制节点某个命令的执行结果的

lookup 插件下面存在很多的方法，这些方式支持通过不同的方式来读取内容作为变量的值

- 1. env 方法：读取控制节点上环境变量的值作为变量的值
- 2. file 方法：读取控制节点上文件的内容作为变量的值
- 3. pipe 方法：读取控制节点上命令的执行结果作为变量的值

lookup 插件所有的方法全部都是读取的控制节点的信息

Markdown 复制

变量名: "{{ lookup('方法'),'方法的值' }}"

env方法:
变量名: "{{ lookup('env'),'PWD' }}"

file方法:
变量名: "{{ lookup('file'),' /etc/passwd' }}"

pipe方法:
变量名: "{{ lookup('pipe'),'echo huawei' }}"

通过 lookup 插件的 pipe 方法，在创建用户的时候实现密码的指定，而不是通过 shell 模块来实现

3.3 自定义 fact 事实变量

自定义的 fact 事实变量，最终会保存到 ansible_facts.ansible_local 中

```
"ipv6": [
  {
    "address": "::1",
    "prefix": "128",
    "scope": "host"
  }
],
"mtu": 65536,
"promisc": false,
"timestamping": [],
"type": "loopback"
},
"ansible_local": {},
"ansible_lsb": {},
"ansible_lvm": {
  "lvs": {},
  "pvs": {},
  "vgs": {}
},
"ansible_machine": "x86_64",
"ansible_machine_id": "e99767a93b914af38fbd61e143c52347",
"ansible_memfree_mb": 888,
"ansible_memory_mb": {
  "nocache": {
    "free": 1202,
```

516,1876%

自定义 fact 事实变量，是在被控节点上定义的，不是在控制节点上定义的

被控节点会在/etc/ansible/facts.d 目录下，去找以.fact 结尾的文件

这个文件就是变量文件，自定义 fact 变量就是写到这个文件中的，采用的定义的格式是 ini 格式

```
[root@adsl-172-10-0-153 ansible]# cat node1.fact
[node1_var]
pkg=nginx
```

```
[root@adsl-172-10-0-153 ansible]# cat node2.fact
[node2_var]
pkg=httpd
```

```
[root@adsl-172-10-0-153 ansible]# ansible all -m shell -a 'mkdir /etc/ansible/facts.d -p'

[root@adsl-172-10-0-153 ansible]# ansible node1 -m copy -a 'src=node1.fact dest=/etc/ansible/facts.d'

[root@adsl-172-10-0-153 ansible]# ansible node2 -m copy -a 'src=node2.fact dest=/etc/ansible/facts.d'
```

4magic 魔法变量

魔法变量是属于特殊的内置变量，这类内置变量具备特殊的含义

内置变量一般都是和配置文件的配置参数相关的

魔法变量最终是在 playbook 引用的，具备一些特殊的意义

比如 groups 魔法变量的值所有的主机组

比如 hostvars 魔法变量

比如 group_names 魔法变量

ansible 的魔法变量，无法通过 ansible 工具查询，只能上 ansible 官网

常见的魔法变量：

1. hostvars：获取指定主机的事实变量

```
"{{ hostvars['node1'].ansible_default_ipv4.address }}"
"{{ hostvars['主机清单中的主机IP或者主机名'].fact事实变量 }}"
```

```
- name: chrony
  hosts: all
  tasks:
- name: tasks
  shell: echo server "{{ ansible_ipv4_default.address }}" iburst >> /etc/chrony.conf
```



echo server "{{ hostvars['node1'].ansible_ipv4_default.address }}" iburst >> /etc/chrony.conf

时间源服务器



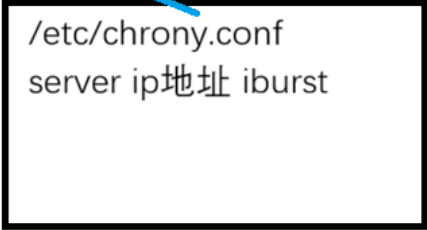
node1



node2



node3



node4

2. inventory_hostname：表示执行任务的主机的主机名

这个主机名到底是主机清单中定义的主机名，还是操作系统本身的主机名？

inventory_hostname 魔法变量—>看的是主机清单中的主机名

ansible_hostname fact 事实变量—>看的被控本身的主机名

配合判断语句来使用：当主机的主机名是 node1 的时候，才会执行这个 task 任务

3. groups：表示主机清单中所有的主机组

JSON | 复制

```
ok: [node1] => {
  "msg": {
    "all": [
      "node1",
      "node2"
    ],
    "servers": [
      "node1",
      "node2"
    ],
    "ungrouped": []
  }
}
ok: [node2] => {
  "msg": {
    "all": [
      "node1",
      "node2"
    ],
    "servers": [
      "node1",
      "node2"
    ],
    "ungrouped": []
  }
}
```

4. group_names：表示运行任务的主机所在的主机组